

Cheat sheet: ranges and algorithms (Session 4)

One page for <ranges> and the algorithm additions since C++17. Badge in brackets is the standard that introduced it. namespace r = std::ranges; namespace v = std::views; throughout.

Vocabulary

- **Range:** anything with begin() and end(). **View:** a range that is cheap to copy and does not own its elements (span, string_view, every views:: result). **Adaptor:** a function that takes a range and returns a view (views::filter). **Sentinel:** whatever end() returns; it no longer has to be the same type as the iterator. **Projection:** a callable applied to each element before the algorithm looks at it (&Record::value). **CPO:** r::begin, r::size, and friends are objects, not functions; they handle arrays, members, and ADL free functions in that order and refuse rvalues of non-borrowed ranges.

Concepts, with one type each

Concept	Means	Example	Not
input_range	single pass	istream_view	
forward_range	multi pass	forward_list, split, join	
bidirectional_range	--	list, filter over a vector	
random_access_range	it + n	deque, iota(0, 10)	filter (predicate must run)
contiguous_range	data()	vector, array, span, string_view	deque
sized_range	O(1) size()	vector, take on a sized range	forward_list, filter, split, istream_view
view	cheap copy, non-owning	span, string_view, adaptor results	vector
borrowed_range	iterators outlive the object	span, string_view, lvalue refs	rvalue vector

Every level includes the ones below. Adapting a vector through filter drops it from random access to bidirectional.

Constrained algorithms [20]

```
r::sort(records, {}, &Record::timestamp);           // range, comparator, projection
r::sort(records, r::greater{}, &Record::value);
auto it = r::lower_bound(records, ts, {}, &Record::timestamp);
auto [min, max] = r::minmax_element(records, {}, &Record::value);
auto [in, out] = r::copy_if(records, std::back_inserter(ok), is_ok); // returns both
```

iterators

```
r::for_each(records, print, &Record::sensor);
```

- Prefer `r::` over `std::` for new code: whole-range overloads, projections, concept errors instead of `enable_if` pages, and they reject rvalue containers (`r::dangling`) instead of `dangling`.
- They return more: `r::copy` gives {in, out}, `r::for_each` gives {in, fun}, `r::minmax` gives {min, max}. Structured bindings are the idiom.
- `std::` still wins for: execution policies (`std::sort(std::execution::par, ...)` has no `ranges::` form), `shift_left` / `shift_right` before C++23, `lexicographical_compare_three_way`.
- A comparator that takes `const Record&` plus a projection is almost always two projections and `{}`: let the algorithm do the member access.

Views and the pipe [20]

```
for (auto& rec : records | v::filter(is_ok) | v::transform(scaled) | v::take(10)) ...  
auto names = records | v::transform(&Record::sensor) |  
r::to<std::vector<std::string>>(); // C++23
```

Adaptor	Does	Notes
<code>filter(p)</code>	keep matches	caches <code>begin()</code> , so not const-iterable; bidirectional at best
<code>transform(f)</code>	map	random access preserved
<code>take(n) / drop(n)</code>	first / all but first n	sized if the source is
<code>take_while(p) / drop_while(p)</code>	until / from first mismatch	forward
<code>reverse</code>	backwards	bidirectional source; costs twice over filter
<code>iota(a, b) / iota(a)</code>	counting source	<code>iota(a)</code> is infinite: pair with <code>take</code>
<code>keys / values / elements<N></code>	tuple member	map iteration
<code>split(delim)</code>	subranges between delimiters	forward only; split of an empty string yields zero pieces, where a hand-written loop usually yields one
<code>join / join_with(d) [23]</code>	flatten	forward only; d may be a range or a single element
<code>all / counted(it, n) / common</code>	wrap	common makes <code>begin/end</code> the same type for old iterator-pair APIs

C++23 adaptors: `zip`, `zip_transform`, `enumerate` (index and element; libc++ 20), `chunk(n)`, `slide(n)`, `stride(n)`, `adjacent<N>` / `pairwise`, `chunk_by(pred)` (group-by on sorted input), `cartesian_product`, `repeat(x)`, `as_rvalue`, `as_const`.

Lazy means: nothing runs until you iterate; each element goes through the whole pipeline before the next starts. `filter` | `transform` | `take` inlines to the loop you would have written.

The rules that bite

1. **Dangling.** A view over an lvalue is a `ref_view`: a pointer. `for (auto& x : load().items())` dies with the temporary before C++23 (P2718, GCC 15 / Clang 19). Name it in the init-statement: `for (auto batch = load(); auto& x : batch.items())`. A view over an rvalue *container* is an *owning_view* and is safe.
2. **const views.** `filter_view`, `drop_while_view`, `split_view` and anything that caches `begin()` are not ranges when `const`. In generic code take `R&& r` and iterate with `auto&&`, not `const R&`.
3. **Cost.** `reverse` of `filter` runs the predicate twice per element. `join` and `split` are forward-only and branchy. Measure before putting them in a hot loop; a plain loop is still right there.
4. **Store containers, not views.** `ranges::to` at function boundaries. A view in a data member is a lifetime bug waiting for a refactor.
5. **Edge cases when converting loops.** Empty input, one element, all-equal keys (for `chunk_by`), empty string (for `split`: zero pieces, not one). Keep the old loop's test.
6. **Concept errors** list every candidate that failed, not the one that would have matched. Read from the bottom.

Folds and the C++23 algorithms

```
auto total = r::fold_left(values, 0.0, std::plus{});           // like accumulate, no init-  
type trap  
auto first = r::fold_left_first(values, r::max);              // optional<T>: empty range  
gives nullopt  
bool any = r::contains(names, "rpm"sv);  
auto last = r::find_last(records, Status::Fault, &Record::status); // subrange from  
the match  
r::iota(buffer, 0);                                           // ranges version of std::iota
```

`fold_left(r, init, op)` with `init` of the right type (0.0, not 0); `fold_right`, `fold_left_with_iter` also exist. `r::starts_with` / `ends_with` (GCC 15). `std::println("{ }", vec)` prints [1, 2, 3] once range formatting lands (GCC 15; libc++ 18).

Parallel algorithms [17]

`std::sort(std::execution::par, v.begin(), v.end())`, also `seq`, `par_unseq`, `unseq` [20]. Only for `std::` iterator-pair algorithms. GCC needs TBB linked for real parallelism; libc++ needs `-fexperimental-library`. Your side of the contract: no data races in the callable, no exceptions escaping (they call `std::terminate`), no mutex inside `par_unseq`. Measure: the crossover is usually tens of thousands of elements, and the shuffle you ran before timing counts if you forgot to reset the clock.

Older additions worth knowing

- **C++17 numerics:** `std::reduce` (unordered accumulate, parallel-friendly), `transform_reduce`, `inclusive_scan` / `exclusive_scan`, `std::gcd`, `std::lcm`, `std::clamp`, `std::sample`; `for_each_n`; `std::search` with `boyer_moore_searcher`.
- **C++20:** `std::shift_left` / `shift_right`, `std::lexicographical_compare_three_way`, `std::erase` / `erase_if`, `r::` versions of nearly every `<algorithm>` function, `std::midpoint`, `std::lerp`.

Monday morning

Constrained algorithms and projections: now, everywhere. Views: in code you own, consumed once. `ranges::to` at boundaries. `clang-tidy modernize-use-ranges` (Clang 19+) converts iterator-pair calls for you. Toolchain gaps per feature: [handouts/toolchain-support-matrix.md](#).